EC991 Topics in Macroeconomics

Week 1 Tutorial · Your computing toolkit

Aseem Patel

University of Essex

Autumn 2026

Five tools, one job each

Tool	What it is	Why it is here
Julia	The programming language	Fast loops, reads like maths, used in the heterogeneous-agent literature
VS Code	The editor	One window for files, REPL, Git and the agent; the Julia extension links them
Git and GitHub	Version control, and a host for it	Your work is never lost; you can read exactly what changed, and who changed it
Pluto	Reactive notebooks	Explore a model with no hidden state; the labs start here
opencode	An AI coding agent	Writes and edits code under your supervision; open source, works with free models

- If you already use another editor, agent or language, you may keep it; the **habits** are what matter
- Everything on this list is free for students; nothing today needs a card

Why learn to code when AI can do it

- A few years ago the hard part was **syntax**: which bracket, which index, why the loop overruns
- Language models have made that part cheap: ask, and you get a value function iteration
- So why a whole tutorial on tooling?

Because the barrier is lower, not gone.

- The agent **writes**; you **decide** whether what it wrote is right
- That decision needs three things: read code fluently, know what the answer should look like, test it
- In this course every number gets checked against something known: a closed form, a theorem, a limit
- An agent cannot do that for you if you do not know what to check

Programming with AI now mostly means you supervise the loop. Learn to supervise it well.

Julia: why this language

Three reasons.

- **Speed where it matters**: a plain `for` loop over a grid is as fast as compiled code
- **Reads like the maths**: Greek letters, `u.(c)` to apply a function to a whole grid
- **The literature uses it**: much current heterogeneous-agent code is in Julia

Present value of a constant income y over $t = 0, \ldots, T$, as you would write it on paper:

```
pv(y; beta = 0.96, T = 10) = sum(beta^t * y for t in 0:T)
pv(1.0; T = 10_000)                # close to 1 / (1 - beta) = 25
```

Python.

- Wherever I can, I provide the **same code in Python**; the week 1 notebook already has a twin
- For the problem sets, **use the language you prefer**; the module outline says so
- The ideas transfer: grids, iteration, interpolation, distributions. The syntax is the small part.

Julia: how it is organised

juliaup installs Julia and keeps it current. Never download Julia from a web page.

The REPL is the interactive prompt. One key at the start of a line switches mode:

Key	Prompt	Mode	Example
`]`	`pkg>`	packages	`add Plots`, `st`
`?`	`help?>`	documentation	`?maximum`
`;`	`shell>`	shell command	`;ls`
`\beta` then Tab	`julia>`	Unicode input	gives β

Environments. Each project records its packages in `Project.toml` and `Manifest.toml`

- `activate .` then `add Plots` or `instantiate`; `st` shows where you are
- **Read the prompt**: `(@v1.12) pkg>` is the global environment, `(ec991) pkg>` is your project
- Nearly every "package not found" error is a package added to the wrong environment

VS Code: one window for everything

- The REPL is where you **experiment**; VS Code is where you **write files**
- The **Julia extension** links the two: cursor on a line, `Shift+Enter`, the line runs in a REPL below
- Open the **folder**, not the file: the extension activates the folder's environment for you

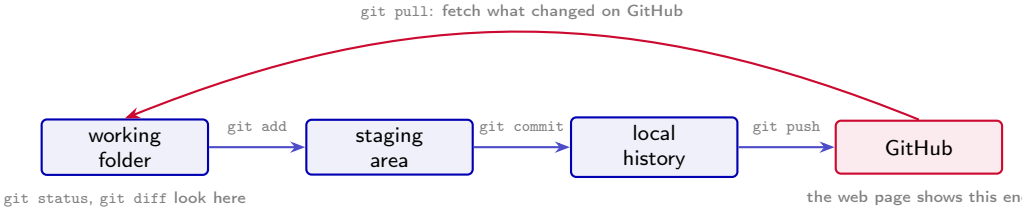
Five shortcuts for day one (Mac: Cmd for Ctrl)

Keys	What it does
`Shift+Enter`	Run the current line or selection in the REPL
`Ctrl+F5`	Run the whole file
`Ctrl+Shift+P`	Command palette: `> Julia: Start REPL`, `> Git: Clone`
`Ctrl+‘`	Integrated terminal (this is where Pluto and opencode run)
`Ctrl+Shift+G`	Source control pane: stage, commit, push without typing Git

Plots open in a pane inside the editor. Hovering over a variable shows its value once the code has run.

Git: a history of your folder

A **commit** is a snapshot with a message, an author and a time. **GitHub** hosts the history online.



Three reasons you want this.

- Your code is **never lost**, and you can always go back to a version that worked
- When an AI agent edits your files, `git diff` shows **exactly what it changed**
- Collaboration and submission run through it; so does the course repository you will clone
- Commit small and often, with a message that says what and why

Pluto notebooks

- A web page of Julia cells; **reactive**: change a cell and every dependent cell re-runs
- No hidden state, so a notebook always means what it shows; good for exploring, poor for long runs
- Packages install themselves and are recorded in the file: anyone who opens it gets the same setup

Three rules that trip up everyone the first time.

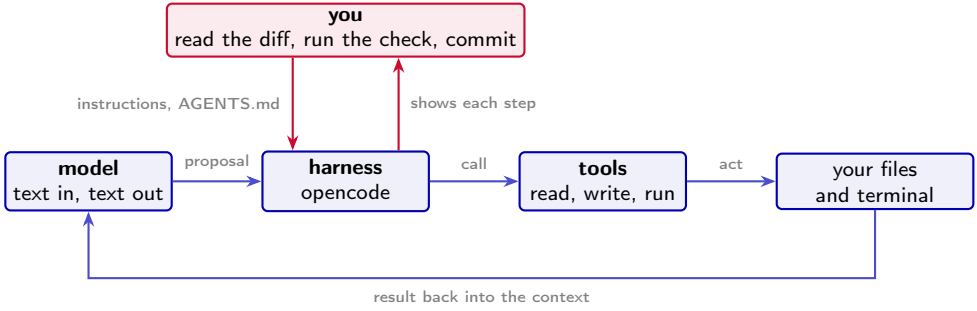
1. **One definition per cell**: to change `x`, edit its cell; do not define it again below
2. Several lines in one cell need `begin ... end` (shared) or `let ... end` (local)
3. **Open the notebook through Pluto, never as a script**

How to start it (from the VS Code terminal): `julia`, then `import Pluto; Pluto.run()`

- A browser tab opens; paste the notebook's path into *Open a notebook*
- Running the .jl file with `Shift+Enter` instead stops with "Package Plots not found": rule 3

An AI coding agent, in four words

An agent is a language model wrapped in a **loop** that can read your files, edit them and run commands.



- **Model**: no memory, no access to your disk. **Harness**: builds the context, runs the tools, shows you.
- **Tools**: the actions it may take. **Instructions**: an `AGENTS.md` that encodes your own rules.
- The model never touches your disk; the harness does, and Git shows you what it did

opencode: how we use it

Set-up. One install command; then `/connect` to pick a model provider, `/models` to pick a model

- **GitHub Copilot** is free with the Student Developer Pack and gives several strong models
- Free alternatives: OpenRouter's `:free` models, Google Gemini's free tier
- An API key is a password that spends money: never paste one into a file, a chat or a notebook

The supervised session you will run today, in your own repository with a clean `git status`:

1. **Plan mode** (press Tab until the footer says *Plan*): the agent proposes, changes nothing
2. Ask for something small and checkable: a function with a test you already know the answer to
3. Read the plan. Does it do **only** what you asked? Then Tab to *Build*
4. **Verify**: `git diff`, run it, read the output; only then `git commit`
5. `/init` writes an `AGENTS.md`; add one rule of your own

Using AI in this module

Allowed, for the problem sets and in the labs, on one condition.

- **Own it**: be able to explain and defend every line you hand in

- The problem sets assess **your** understanding of the models and methods, not the agent's
- Code you cannot explain when asked in a tutorial is treated as not your work
- The habits from the last two slides are how you stay on the right side of this

Now it is your turn

Open the page and work through it **in order**. Each part ends with something you can see working.

- Every course file, this page included, is on **Moodle** and on the course **GitHub** repository

Part	You install or do	You know it worked when
1	Terminal, Git, GitHub account	`git -version` prints a version
2	Julia via juliaup; add `Pluto`	`1 + 1` in the REPL; `juliaup status`
3, 4	VS Code, Julia extension, a project	`Shift+Enter` runs a line; `Project.toml` appears
5	Smoke test `week01_check.jl`	Seven `PASS` lines
6	`ec991-work` repository	Your commit is visible on GitHub
7, 8	Pluto and the Julia-basics notebook	Exercises 2, 4 and 5 turn green
9	opencode, provider, supervised session	A reviewed diff, committed; `AGENTS.md` exists
11	Checklist	Every box ticked

- Stuck for more than a moment? Check the **troubleshooting table** in Part 11, then call me over
- Finished early? Sections 6 to 8 of the notebook, then the Julia and Python table in Part 8

The errors you will meet

Symptom	Cause and fix
`julia: command not found`	The terminal was open before the install. Close it and open a new one.
`Package Plots not found`, in a script	Wrong environment active. `]` `activate .` then `instantiate` or `add Plots`.
`Package Plots not found`, on the notebook	You ran the notebook as a script. Open it through Pluto.
Pluto: *Multiple definitions for x*	The same name is defined in two cells. Delete one.
Pluto: *Multiple expressions in one cell*	Wrap the cell in `begin ... end` or `let ... end`.
The first plot takes a long time	Normal: Julia compiles Plots on first use. The second plot is instant.
Git refuses to commit	Set `user.name` and `user.email` (Part 1).
opencode: *no provider configured*	Run `/connect` again; `opencode auth list` shows stored credentials.

For week 2

Finish the notebook. Sections 6 to 8: plotting, a slider, timing your code and testing it against a closed form.

Pull the course repository on GitHub (link on Moodle): `git pull` in its folder, then `] instantiate`. Every file is on both.

What the lab assumes.

- You can write a function with keyword arguments and apply it to a grid with a dot
- You can write `for t in T:-1:1` and a `while` loop with a tolerance
- You know `maximum`, `argmax`, and `c > 0 ? u(c) : -Inf`
- Everything on this list is in the notebook, and its three check cells tell you when you have it